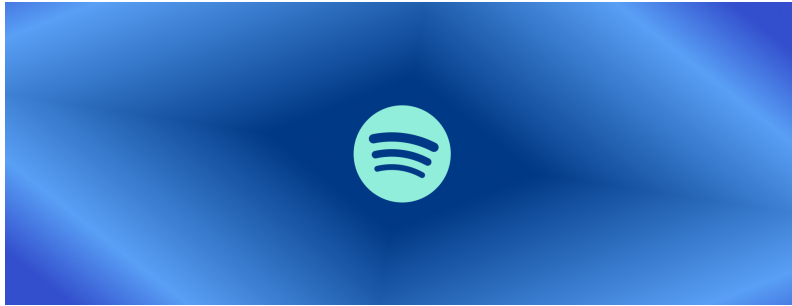


Sujet 3

Spotify Playlist Manager

Bibliothèque musicale personnelle avec playlists



Rappel : ce document décrit UNIQUEMENT le sujet. Toutes les contraintes techniques (architecture, tris, pagination, remise, barème) sont dans le document **Consignes_Generales_Lab2.pdf**. Lisez les DEUX avant de commencer.

1. Contexte

Spotify, Apple Music, Deezer, YouTube Music : les plateformes de streaming musical permettent aux utilisateurs de **parcourir** un catalogue immense, **créer** leurs propres playlists, **filtrer** par genre ou artiste, et **rechercher** n'importe quoi en temps réel.

Votre mission : construire une application locale qui gère un catalogue de **300 à 500 chansons**, plus un système de **playlists personnelles**.

2. Vocabulaire métier

Terme	Définition
Chanson (Track)	Une piste audio individuelle
Artiste	Interprète (peut être un groupe ou une personne)
Album	Collection de chansons publiées ensemble
Genre	Catégorie musicale (Pop, Rock, Hip-Hop, Jazz, Classique, EDM...)
Durée	Longueur de la chanson (en secondes ou format mm:ss)
Playlist	Sélection de chansons créée par l'utilisateur
Bibliothèque	Ensemble des chansons disponibles dans l'application
File d'attente (Queue)	Chansons à venir dans la lecture
Écoutes	Nombre de fois qu'une chanson a été jouée (compteur simulé)

3. Concepts à modéliser

3.1 Chanson

La classe centrale avec au minimum :

- Identifiant unique
- Titre
- Nom de l'artiste
- Nom de l'album

- Année de sortie
- Genre (enum)
- Durée en secondes
- Nombre d'écoutes (int)

3.2 Playlist

Une classe qui représente une playlist personnelle :

- Nom (donné par l'utilisateur)
 - Liste ordonnée de chansons
 - Date de création
 - Durée totale (calculée)
- Opérations attendues sur une playlist :
- Ajouter une chanson (pas de doublon)
 - Retirer une chanson
 - Réordonner (déplacer une chanson vers le haut ou le bas)
 - Vider entièrement

3.3 Genre

Utilisez un enum pour représenter les genres musicaux (Pop, Rock, HipHop, Jazz, Classique, Electronic, Metal, Country, RnB, Reggae...).

3.4 Bibliothèque

Une classe agrégateur qui contient toutes les chansons et toutes les playlists créées par l'utilisateur.

4. Fonctionnalités spécifiques au sujet

4.1 Critères de tri à supporter (au moins 4 parmi ces 6)

- Titre alphabétique
- Artiste alphabétique
- Durée croissante (courte au long)
- Année (plus récent en premier ou l'inverse)
- Nombre d'écoutes décroissant (les plus écoutées en premier)
- Genre (regroupement alphabétique)

4.2 Filtres qui se combinent (au moins 4 parmi ces 6)

- Genre (ComboBox)
- Décennie (ComboBox : Toutes / 1970s / 1980s / ... / 2020s)
- Artiste spécifique (ComboBox avec tous les artistes)
- Durée maximum (Slider)
- Nombre d'écoutes minimum (Slider ou Spinner)
- Chansons dans une playlist donnée (afficher seulement les chansons d'une playlist choisie)

4.3 Recherche par texte

Un champ de recherche filtre en temps réel selon :

- Le titre de la chanson
 - Le nom de l'artiste
 - Éventuellement l'album (bonus)
- Insensible à la casse et aux accents.

4.4 Gestion de playlists

L'application doit permettre :

- Créer une playlist (avec nom)
- Supprimer une playlist
- Ajouter la chanson sélectionnée dans une playlist choisie (ComboBox ou dialog)
- Retirer une chanson d'une playlist
- Voir le contenu d'une playlist (liste + durée totale)

4.5 Lecteur simulé

Un panneau en bas qui affiche :

- La chanson en cours (titre + artiste)
- Boutons Play / Pause / Next / Previous (simulés, pas d'audio réel obligatoire)
- Slider de progression factice
- Bouton Shuffle (utilise `Random`)

Cliquer sur "Next" simule qu'une chanson a été écoutée : le compteur d'écoutes s'incrémente et la chanson suivante devient "en cours".

5. Sources de données possibles

Pour votre CSV :

- **Dataset Spotify** sur Kaggle : "Spotify Top Tracks" (contient déjà titre, artiste, durée, popularité)
- **Générer avec Python** : liste d'artistes réels × titres inventés × durées cohérentes (2-6 min)
- **Combiner** : 100 vraies chansons connues + 300 générées

Minimum 300 chansons, maximum 500. Utilisez au moins 8 genres différents et 30 artistes distincts.

Champs suggérés : `id`, `titre`, `artiste`, `album`, `annee`, `genre`, `duree_sec`, `ecoutes`

6. Bonus spécifiques au sujet (facultatifs)

- **Mode Shuffle** : le bouton Shuffle joue les chansons dans un ordre aléatoire
- **Statistiques d'écoute** : afficher le top 10 des chansons les plus écoutées, le genre le plus écouté
- **Import/Export de playlist** en fichier JSON
- **Auto-création** de playlist "Mix quotidien" : 20 chansons choisies aléatoirement parmi les préférées de l'utilisateur

- **Formatage des durées** : afficher 3 :45 au lieu de 225 secondes
- Trois **thèmes** visuels : sombre (Spotify), clair, personnalisé

7. Cas de test à valider avant la remise

1. Charge sans erreur 300+ chansons
2. Tri par durée croissante : la plus courte en premier
3. Tri par nombre d'écoutes décroissant : la plus écoutée en premier
4. Filtre "genre = Rock" affiche uniquement du Rock
5. Combinaison "Rock + 2010s" fonctionne
6. Recherche "beatles" trouve tous les Beatles (insensible casse)
7. Créer une playlist "Mon top", ajouter 5 chansons, la voir dans le menu
8. Ajouter deux fois la même chanson à une playlist ne crée pas de doublon
9. Retirer une chanson de la playlist met à jour l'affichage
10. Pagination : navigation 25 par page
11. Le bouton Next simule l'écoute (compteur augmente)

8. À retenir

Ce sujet, en une phrase : construire un gestionnaire de bibliothèque musicale avec un système riche de playlists (créer, remplir, réordonner, supprimer).

Le point unique de ce sujet : la GESTION dynamique de playlists — pas juste une wishlist, mais plusieurs listes nommées que l'utilisateur crée et modifie.

Bon développement !